

Emelt digitális kultúra – SZÓBELI felkészítő

A) témakörök · B) programozás · mintatételek · gyakorlási terv · szóbeli: 2026. november 7., 09:00, Zalaegerszeg

Hogyan használd ezt a segédletet

- Az írásbeli segédlet (Excel, SQL, C++) **kiegészítése**: a függvénylistákra és a C++ kódmintákra nem térek ki újra, ott hivatkozom rájuk.
- A tételsor **nem nyilvános**, csak a témakörök. A mintatételek **saját készítésűek, nem hivatalosak**: arra valók, hogy begyakorold a szerkezetet, nem arra, hogy a vizsgán pont ezeket kapd.
- A menüneveket magyar nyelvű Office 2021-re (Word, Excel, PowerPoint) írtam, a GIMP és az Inkscape menüit angolul adom meg. **A vizsgagépen ellenőrizd őket**, LibreOffice-ban és más nyelvű felületen eltérhetnek. Ahol bizonytalan vagyok, „(ellenőrizd)” áll mellette.
- A segédlet **Word-fájl, szabadon szerkesztheted**: írd át a saját szavaidra. A szóbelin a saját fogalmazásod hitelesebb, mint egy bemagolt szöveg.
- Minden A) fejezet ugyanúgy épül fel: **tételvázlat → gépes bemutatók → szakkifejezések → bizottsági kérdések → gyakori hibák**.

Tartalom

Fejezet	Mit találsz benne
1. Kisokos	A vizsga felépítése és pontozása, a 30 perces felkészülés és a 20 perces felelet időbeosztása, nyitómondat, kommunikáció
2. Szövegszerkesztés	Stílusok, hosszú dokumentum, körlevél, változáskövetés, tipográfia (Word)
3. Grafika és képszerkesztés	Pixel- és vektorgrafika, színmodellek, formátumok, GIMP és Inkscape
4. Bemutatókészítés	Diaminta, tervezési elvek, animáció és átmenet, kimenetek (PowerPoint)
5. Publikálás a világhálón	HTML5, CSS, szelektorok, doboz-modell, reszponzív oldal, webes alapfogalmak
6. Táblázatkezelés	Hivatkozások, függvények, kimutatás, diagram, feltételes formázás (Excel 2021)
7. Adatbázis-kezelés	Relációs modell, normalizálás, SQL, phpMyAdmin (MariaDB)
8. B) Programozás	Módszer, pszeudokód ↔ C++, programozási tételek kóddal és szóbeli magyarázattal
9. B) Mintafeladatok	7 kidolgozott feladat és 3 kódértelmezési feladat
10. Gyakorlás	12 mintatétel (A + B), értékelőlap, napi terv november 7-ig, vizsganapi lista

A 2026-os vizsgakörnyezet (hivatalos szoftverlista alapján)

Terület	Program	Megjegyzés
Irodai csomag	Microsoft Office Professional Plus 2021 vagy LibreOffice 25.2	A vizsgázó választ. A menük és néhány függvénynév eltér.
Adatbázis (emelt)	MariaDB 10 + phpMyAdmin 5.2 (XAMPP)	Emelt szinten nincs Access vagy LibreOffice Base. MySQL-kompatibilis.
Grafika	GIMP 3.0 (pixel), Inkscape 1.4 (vektor)	Mindkettő szerepel a listán.
Weblap	Visual Studio Code vagy Notepad++	HTML és CSS szerkesztésére.
Programozás	Code::Blocks (C/C++), Python 3.13 + IDLE, Visual Studio, NetBeans, IntelliJ	Neked: C++ / Code::Blocks .
Offline súgó	Zeal (C++, CSS, HTML, MySQL dokumentáció)	A gyakorlati vizsgán használható internet nélkül.

Figyelem: a szóbeli **Zalaegerszegen** lesz, nem a saját iskoládban. Tisztázd előre (az iskolában vagy a jelentkezéskor megadott szoftvernyilatkozat alapján), hogy ott Windows vagy Linux gép, Office vagy LibreOffice lesz, és melyik programokat érheted el. A források szerint a szóbelin két gép áll rendelkezésre: egy internetes és egy internet nélküli. Szakmai kérdéssel az OH-t is megkeresheted: digitalis_kultura@oh.gov.hu.

1. Kisokos – a szóbeli menete

Felépítés · pontozás · időbeosztás · nyitómondat · kommunikáció

A vizsga felépítése

Elem	Amit tudni érdemes
Időtartam	30 perc felkészülés , majd legfeljebb 20 perc felelet .
Tétel	Legalább 20 tétel, minden témakörből szerepel tétel. Egy tétel két részből áll: A) téma kifejtése + B) programozási feladat.
A) rész	Egy téma szóbeli kifejtése konkrét tartalmi szempontok alapján , kérésre számítógépes szemléltetéssel. Témakörök: szövegszerkesztés, grafika és képszerkesztés, bemutatókészítés, publikálás a világhálón, táblázatkezelés, adatbázis-kezelés.
B) rész	Egy vagy több programozási feladat megoldása és bemutatása: kódolás, kódmódosítás vagy kódértelmezés (pszeudokód is lehet). A B) feladatot a felkészülési időben kell megoldanod , a felelet alatt bemutatsz.
Eszközök	Segédeszközt nem kell hozni. A felkészülés alatt jegyzetet készíthetsz. Kódoláshoz internet nélküli gép áll rendelkezésre.
Hivatalos anyag	A tételek nem nyilvánosak, csak a témakörök. A követelményeket az Oktatási Hivatal vizsgaleírása tartalmazza.

Pontozás

Szempont	Pont	Mit néznek
Tartalom: A) feladat	8	Mindent tartalmaz-e, ami a témához kell; nincsenek-e tárgyi tévedések, rossz magyarázatok
Tartalom: B) feladat	10	Működik-e a megoldás, helyes-e az algoritmus, érted-e, amit írtál
Logikai felépítés	4	Jó időbeosztás, a lényeg kiemelése, követhető gondolatmenet
Kifejezőképesség, szaknyelv	4	Pontos szakkifejezések, érthető megfogalmazás
Kommunikatív készség	4	Lehet-e vezetni; megérted-e a kérdést; lehet-e tartalmas párbeszédet folytatni

A rubrika összege $18 + 4 + 4 + 4 = 30$ pont. Egyes oldalak más összpontszámot írnak: a pontos értékelést a hivatalos útmutatóban ellenőrizd. A lényeg mindenképp ugyanaz: a pontok több mint fele a tartalomra megy, a többi a **hogyan mondd el** kategóriákból jön.

A 30 perc felkészülés – ajánlott beosztás

Idő	Teendő
0–3 perc	Olvasd el a tételt. Az A) rész tartalmi szempontjait számozd meg a lapon. A B) részből írd ki: mi a bemenet , mi a kimenet , milyen megkötések vannak.
3–18 perc	B) feladat gépen. Először egy kis példa kézzel, majd a programozási tétel azonosítása, kód, három teszt (alap, szélső eset, üres/egyelemű). Mentsd el, és hagyd megnyitva.
18–26 perc	A) vázlat a jegyzetlapra: szempontonként 1–2 kulcsszó + 1 gépes bemutató, amit meg tudsz mutatni. Ne mondatokat írd.
26–30 perc	Mondd el magadban a nyitómondatot és a B) magyarázat első két mondatát. Ellenőrizd, hogy a program fut, és a fájlok megnyithatók.

Ha a B) feladat nem megy: ne állj le vele. Írd le pszeudokódban, hogy mit kellene csinálnia, és jelöld meg, hol akadsz el. Így a megértésért és a magyarázatért is kaphatsz pontot. Az A) részt ilyenkor is szánd rá a teljes időt.

A 20 perc felelet – időterv

Perc	Mit csinálsz
0:00–0:30	Köszönés, nyitómondat (lent). A bizottság látja, hogy tudod, mi jön.
0:30–9:00	A) rész: a szempontokat sorban végigveszed. Kérésre megmutatod a gépen. Jelezd: „Ezt meg tudom mutatni a gépen.”
9:00–17:00	B) rész: feladat → ötlet (melyik programozási tétel) → kód → futtatás → tesztek → lehetséges módosítás.
17:00–20:00	Kérdések, kiegészítés, egy mondatos összegzés. A bizottság elég gyakran az utolsó percekben kérdez vagy módosítat.

Nyitómondat – sablon

„Tisztelt Bizottság! A tételem két részből áll. Az **A) részben** a(z) **[témakör]** témaköréről beszélek, és ezeket a szempontokat érintem: **[1., 2., 3.]**. A **B) részben** egy programozási feladatot oldottam meg, amely **[egy mondatban a feladat]**. Az A) résszel kezdem.”

Kommunikáció – mit csinálj és mit ne

- **Szerkezetet jelezz:** „Először..., másodszor..., végül...”. Ez a logikai felépítés 4 pontja, és az idődet is kézben tartja.
- **Pontos szakszó, utána egyszerű magyarázat:** „A stílus egy elnevezett formázási készlet, vagyis egyszerre módosíthatok vele minden címsort.”
- **Kérdésre rövid, célzott válasz.** Ha nem érted, Kérd meg a bizottságot: „Megismételné, kérem?” vagy „Úgy értette, hogy...?”.
- **Ha elakadsz,** mondd el, amit biztosan tudsz: „Ezt pontosan nem tudom, de annyit tudok, hogy...”. A hallgatás és a találgatás kijelentő módban a legrosszabb.
- **A gépnél hangosan gondolkodj:** „Most kijelölöm a címsorokat, mert a tartalomjegyzék ebből dolgozik.” Így a bizottság követi, mit és miért teszel.
- **Ne olvasd fel a jegyzetet,** a vázlat csak kapaszkodó. Nézz a bizottságra, ne a lapra.

2. Szövegszerkesztés (Word 2021)

Stílusok · hosszú dokumentum · körlevél · változáskövetés · tipográfia

A mondandó gerince (tételvázlat, ellenőrzőlista)

- **Mit jelent?** Szöveges dokumentumok létrehozása, formázása, tárolása és nyomtatása. A **tartalmi** (szöveg, ábra, táblázat) és a **formai** réteget szétválasztjuk, ezt a **stílusok** teszik lehetővé.
- **Formázási szintek:** karakter (betűtípus, méret, dőlt, aláhúzás), bekezdés (igazítás, behúzás, sorköz, térköz, tabulátor, szegély, felsorolás), oldal/szakasz (méret, tájolás, margó, hasábok, élőfej/élőláb, oldalszámozás).
- **Stílusok:** bekezdés- és karakterstílus. A stílus **öröklődik** (alapja), és a módosítása mindenhol átvezetődik. A **Címsor 1–3** stílusok adják a tartalomjegyzék és a navigációs ablak alapját.
- **Hosszú dokumentum:** tartalomjegyzék (mező, frissíteni kell), többszintű lista, oldal- és szakasztörések, élőfej/élőláb (előző szakaszhoz kapcsolás), oldalszámozás, keresztbehivatkozás, képaláírás, ábrajegyzék, lábjegyzet, irodalomjegyzék.
- **Táblázat és kép:** táblázat szerkezete (cellaegyesítés, felosztás, ismétlődő fejlécsor), rendezés; kép arányos átméretezése, **szövegtördelés** (sorban, négyzetes, szoros, szöveg mögött).
- **Körlevél:** törzsdokumentum + adatforrás (táblázat, adatbázis) + egyesítési mezők; szűrés, rendezés; az eredmény új dokumentum vagy nyomtatás.
- **Együttműködés és ellenőrzés:** változások követése, megjegyzések, dokumentumok összehasonlítása, helyesírás-ellenőrzés (**nyelv beállítása!**), védelem.
- **Tipográfia, helyesírás:** nem törhető szóköz a szám és a mértékegység között, gondolatjel (–) és kötőjel (–) különbsége, magyar idézőjel („ ”), térköz az üres sor helyett, tabulátor a szóközös igazítás helyett, árva és özvegy sorok kerülése, hosszú szöveghez talpas betű.
- **Mentés, formátum:** .docx, .dotx (sablon), .pdf (megőrzi a megjelenést), .odt, .txt (formázás nélkül); oldalbeállítás és nyomtatási kép.

Gépes bemutatók (kérésre, lépésről lépésre)

Bemutató	Lépések
Stílus módosítása	Kezdőlap → Stílusok → jobb gomb a Címsor 1 -en → Módosítás: betűtípus, térköz, a következő bekezdés stílusa. Mutasd meg, hogy minden Címsor 1 egyszerre változik.
Tartalomjegyzék	Címsor stílusok alkalmazása → Hivatkozások → Tartalomjegyzék → Automatikus. A szöveg módosítása után: jobb gomb → Mező frissítése (F9).
Szakasztörés, eltérő élőfej	Elrendezés → Törések → Következő oldal . Az új szakasz élőfejen duplán kattintva kapcsold ki az Előző szakaszhoz kapcsolás gombot. A címlapra: Eltérő első oldal . Oldalszám: Beszúrás → Oldalszám, formátum: kezdő sorszám.
Többszintű lista a címsorokhoz	Kezdőlap → Többszintű lista → Új többszintű lista meghatározása → a szintek összekapcsolása a Címsor 1–3 stílusokkal (1., 1.1., 1.1.1.).
Körlevél	Levelezések → Körlevél indítása → Címzettek kiválasztása (Excel/CSV) → Egyesítési mező beszúrása → Eredmények előnézete → Egyesítés befejezése.
Változások követése	Korrektúra → Változások követése. Módosítsd a szöveget, majd Elfogadás / Elvetés , megjegyzés beszúrása. Nézd meg a Dokumentumok összehasonlítása funkciót is.
Hasábok és tabulátorok	Elrendezés → Hasábok (a kijelölt szövegre, vagy szakaszra). Tabulátor: vonalzon kattintás, vagy Bekezdés → Tabulátorok (jobbra zárt, tizedes, kitöltő karakter).

Szakkifejezések

Fogalom	Magyarázat	Fogalom	Magyarázat
Bekezdés	Az Entertől Enterig tartó szövegegység, a bekezdésformázás az egészre vonatkozik.	Stílus	Elnevezett formázási készlet, egyben alkalmazható és módosítható.
Szakasz	Azonos oldalbeállítású dokumentumrész (tájolás, hasábok, élőfej); szakasztörés választja el.	Élőfej / élőláb	Az oldal felső / alsó margójában ismétlődő tartalom (cím, oldalszám).
Mező	Automatikusan frissülő elem: oldalszám, dátum, tartalomjegyzék.	Tabulátor	Oszlopszerű igazítási pont: bal, jobb, középre, tizedes; lehet kitöltő karakterrel.
Behúzás	A bekezdés eltolása a margótól: bal, jobb, első sor, függő.	Szövegtördelés	A kép és a szöveg viszonya: sorban, körbefutás, mögött.
Körlevél	Személyre szabott levelek törzsdokumentumból és adatforrásból.	Sablon	Előre formázott minta (.dotx), amiből új dokumentum készül.
Nem törhető szóköz	Olyan szóköz, amelynél a sor nem törhet (Ctrl+Shift+Szóköz).	Árva / özvegy sor	A bekezdés első / utolsó sora egyedül marad az oldal alján / tetején.

Bizottsági kérdések – rövid mintaválaszok

Kérdés	Rövid válasz
Miért jobb stílust használni, mint kézzel formázni?	Egységes lesz a dokumentum, egy helyen módosítható, a címsorokból automatikus tartalomjegyzék és navigáció készül, és kevesebb a hiba.
Mi a különbség az oldaltörés és a szakasztörés között?	Az oldaltörés csak új oldalt kezd. A szakasztörés új szakaszt nyit, amelynek külön oldalbeállítása (tájolás, hasáb), élőfeje és számozása lehet.
Bekezdés- és karakterstílus különbsége?	A bekezdésstílus a teljes bekezdést formázza (igazítás, térköz, betű), a karakterstílus csak a kijelölt szövegrészt (pl. kiemelés).
Hogyan lesz automatikus a tartalomjegyzék?	A címetek Címsor stílussal látjuk el, a tartalomjegyzék mező ezekből épül. Módosítás után frissíteni kell.
A címlapon ne legyen oldalszám. Hogyan?	Eltérő első oldal beállítással vagy külön szakasszal, ahol az előző szakaszhoz kapcsolást kikapcsoljuk.
Mire jó a körlevél?	Egy törzsdokumentumból és egy adatforrásból személyre szabott leveleket készít (név, cím, tartalmi mezők).

Gyakori hibák

- Üres bekezdésekkel, kézi sortöréssel vagy szóközökkel „igazítani”: térközt, behúzást, tabulátort használj.
- Kézzel számozott címsorok és kézzel írt tartalomjegyzék: ezek nem frissülnek.
- A tartalomjegyzék frissítésének elfelejtése; a nyelv nincs beállítva, ezért a helyesírás-ellenőrző téves hibákat jelez.

3. Számítógépes grafika és képszerkesztés (GIMP 3.0, Inkscape 1.4)

Pixel- és vektorgrafika · színmodellek · formátumok · rétegek · átlátszóság · export

A mondandó gerince (tételvázlat, ellenőrzőlista)

- **Két alaptípus. Pixelgrafika** (raszteres): képpontok rácsa, fényképekhez jó, felbontásfüggő, átméretezéskor minőséget veszít. **Vektorgrafika**: alakzatok és görbék (Bézier) matematikai leírása, felbontásfüggetlen, logóhoz és ábrákhoz jó.
- **Képjellemzők**: felbontás (ppi/dpi), képméret képpontban, **színmélység** (bit/képpont: 1, 8, 24, 32), színmodellek: **RGB** (additív, kijelző), **CMYK** (szubtraktív, nyomtatás), HSV/HSL, szürkeárnyalat; **alfa csatorna** (átlátszóság).
- **Fájl méret számítása** (tömörítetlen): szélesség × magasság × bitmélység / 8 bájt. Példa: 1920 × 1080 képpont, 24 bit → 1920 · 1080 · 3 = 6 220 800 bájt ≈ 5,9 MiB.
- **Formátumok**: BMP (tömörítetlen), JPEG (veszteséges, fotó, nincs átlátszóság), PNG (veszteségmentes, átlátszóság, grafikák), GIF (max. 256 szín, animáció), TIFF (nyomda), SVG (vektor, XML-alapú), XCF (GIMP natív, rétegekkel).
- **Pixelgrafikus szerkesztés (GIMP)**: rétegek (sorrend, átlátszóság, keverési mód), kijelölések (téglalap, ellipszis, szabadkézi, varázspálca, színtartomány), rétegmaszk, átméretezés, vászonméret, vágás, színkorrekció (fényerő/kontraszt, szintek, görbék), retusálás (klónozás), szűrés, szöveg.
- **Vektorgrafikus szerkesztés (Inkscape)**: alapalakzatok, útvonalak és csomópontok, kitöltés és körvonal, színátmenet, csoportosítás és rétegek, igazítás/elosztás, szöveg útvonallá alakítása, útvonal-műveletek (egyesítés, kivonás, metszet).
- **Melyiket mikor?** Fotó → pixelgrafika. Logó, ikon, műszaki ábra, ami többféle méretben kell → vektorgrafika. Vektorból pixelkép könnyen készül (rasztrózás), fordítva nehéz (vektoralás).
- **Jog és forrás**: szerzői jog, Creative Commons licenc, forrásmegjelölés.

Gépes bemutatók (kérésre, lépésről lépésre)

Bemutató	Lépések
Háttér eltávolítása (GIMP)	Layer → Transparency → Add Alpha Channel . Varázspálca (Fuzzy Select) vagy Foreground Select a háttérre → Delete. Exportálás PNG -ként (átlátszóság megmarad); JPEG nem tud átlátszóságot.
Átméretezés (GIMP)	Image → Scale Image (a láncszem az arányt rögzíti, interpoláció). Különböztessd meg: Image → Canvas Size (a vászon nő, a kép nem) és Image → Crop to Selection (vágás).
Rétegek és szöveg (GIMP)	Két kép összeszerkesztése külön rétegen; réteg átlátszósága és keverési módja; szöveg eszköz (új szövegréteg); mentés XCF -be, kész kép exportja: File → Export As (JPEG minőség csúszkával).
Színkorrekció (GIMP)	Colors → Levels / Curves / Brightness-Contrast / Hue-Saturation. Mutasd meg az előtte–utána különbséget, és hogy melyik réteg dolgozik.
Rajz készítése (Inkscape)	Alakzatok rajzolása, kitöltés és körvonal (Fill and Stroke), csomópont-szerkesztés (Node tool), színátmenet, csoportosítás (Ctrl+G), igazítás (Align and Distribute).
Szöveg és export (Inkscape)	Szöveg beszúrása → Path → Object to Path (útvonallá alakítás, betűtípus nélkül is megmarad). Mentés SVG-ként, exportálás PNG-be adott felbontással (File → Export).
Vektor vs pixel	Nagyíts rá erősen egy pixelképre és egy SVG-re: a pixelkép „kockássá” válik, a vektor éles marad.

Szakkifejezések

Fogalom	Magyarázat	Fogalom	Magyarázat
Pixel	A raszteres kép legkisebb eleme, egy színértékkel.	Felbontás	Képpont/hüvelyk (ppi, dpi); nyomtatáshoz általában nagyobb kell, mint képernyőre.
Színmélység	Egy képpont színének tárolásához használt bitek száma (24 bit = ~16,7 millió szín).	RGB / CMYK	RGB: fénnel keverés (kijelző). CMYK: festékkel keverés (nyomtatás).
Alfa csatorna	Az átlátszóságot tároló extra csatorna (pl. PNG).	Réteg	Átlátszó fólia-szerű szint, külön szerkeszthető.
Kijelölés / maszk	A kép egy részének védelme vagy elkülönítése a szerkesztéshez.	Interpoláció	Átméretezéskor az új képpontok színének kiszámítása.
Veszteséges tömörítés	Információt dobunk el a kisebb méretért (JPEG); veszteségmentesnél nem (PNG).	Bézier-görbe	Vektoros görbe, amit kontrollpontok és csomópontok határoznak meg.
Rasztrózás	Vektoros kép átalakítása képpontokká (pl. PNG-exportkor).	Vektoralás	Pixelkép átalakítása vektorossá (nehéz, nem mindig pontos).

Bizottsági kérdések – rövid mintaválaszok

Kérdés	Rövid válasz
Miért romlik a pixelkép minősége nagyításkor?	A képpontok száma fix, nagyításkor új pontokat számol ki az interpoláció, így a kép elmosódik vagy kockássá válik.
JPEG vagy PNG a weblapra?	Fotóhoz JPEG (kisebb), logóhoz, ábrához és átlátszósághoz PNG (veszteségmentes, alfa csatorna).
Miért CMYK a nyomtatásban?	A festékek a fényt elnyelik (szubtraktív keverés), ezért a nyomtató C, M, Y, K színeivel dolgozik.
Képméret vagy vászonméret átméretezése?	A képméret a tartalmat is skálázza, a vászonméret csak a kép körüli területet változtatja.
Mit jelent a veszteséges tömörítés?	A tömörítés során információ vesz el, ami nem állítható vissza (ismételt JPEG-mentésnél romlik a kép).
Hogyan számolod ki egy kép méretét?	Képpontok száma × bitmélység / 8 bájt, tömörítés nélkül; a tömörített fájl kisebb.

Gyakori hibák

- Kis kép mesterséges nagyítása („javítás”): ettől nem lesz éles, csak nagyobb.
- JPEG többszöri mentése: minden mentésnél romlik a minőség. Dolgozz XCF-ben vagy PNG-ben, a végén exportálj.
- Átlátszó háttér JPEG-ben: nem lehet, PNG kell.
- Arányok torzítása átméretezéskor (a lánc kikapcsolva).
- Rétegek idő előtti összeolvasztása: utólag nem lehet javítani.

4. Bemutatókészítés (PowerPoint 2021)

Diaminta · tervezési elvek · átmenet és animáció · vetítés · kimenetek

A mondandó gerince (tételvázlat, ellenőrzőlista)

- **Cél és közönség.** A bemutató a mondandót támogatja, nem helyettesíti. Szerkezet: bevezetés, tárgyalás, összegzés; **egy dia – egy gondolat.**
- **Egységes arculat: diaminta** (Nézet → Diaminta), elrendezések, téma (színek, betűk, effektek), helyőrzők. A diaszám, dátum, élőláb a mintán állítható.
- **Tartalmi elemek:** szöveg (kulcsszavak, listaszintek), kép, ábra, **SmartArt**, táblázat, diagram, hang, videó, hiperhivatkozás és műveleti gomb.
- **Átmenet és animáció.** Az **átmenet** a diák közti váltás, az **animáció** a dián belüli objektumok mozgása (belépés, kiemelés, kilépés, mozgásút). Indítás: kattintásra, előzővel, előző után. Mértékkel használd.
- **Vetítés:** előadói nézet, jegyzetek, rejtett dia, szakaszok, időzített vagy folyamatos vetítés, egyéni vetítés.
- **Tervezési elvek:** nagy kontraszt, olvasható betűméret (törzsszövegnél kb. 24 pt vagy nagyobb), kevés szöveg, legfeljebb két betűtípus, 3–4 fő szín, jó minőségű képek, ábra a szöveg helyett, **ne olvassd fel a diát.**
- **Akadálymentesség:** alternatív szöveg a képeken, kontraszt, az olvasási sorrend ellenőrzése, érthető dia-címek.
- **Kimenetek:** .pptx (szerkeszthető), .ppsx (vetítés), PDF, jegyzetoldal / kiosztandó nyomtatása, videó exportálása.
- **Forrás és jog:** képek és zenék licence, forrásmegjelölés a végén.

Gépes bemutatók (kérésre, lépésről lépésre)

Bemutató	Lépések
Diaminta módosítása	Nézet → Diaminta . A főmintán állítsd be a háttért, betűtípust, logót, diaszámot; zárd be a mintanézetet. Mutasd meg: minden dián megjelenik, új dián is.
Téma és elrendezés váltása	Tervezés → téma és változat; Kezdőlap → Elrendezés (Címdia, Cím és tartalom, Két tartalom). Mutasd meg, hogy az elrendezés a diamintából jön.
SmartArt	Beszúrás → SmartArt , vagy a meglévő felsorolás átalakítása SmartArt-grafikává (ellenőrizd a menüpont nevét). Szint és szín módosítása.
Átmenet és animáció	Átmenetek fül: dia-váltás effektje és időtartama. Animációk fül: objektum kijelölése → belépő animáció → Animációs ablaktábla → Indítás: kattintásra / előző után.
Hivatkozás, műveleti gomb	Beszúrás → Hivatkozás (másik diára, weboldalra), Beszúrás → Alakzatok → Műveleti gomb (ugrás a tartalomjegyzék-diára).
Előadói nézet és export	Diavetítés → Előadói nézet (jegyzetek, óra, következő dia). Fájl → Exportálás → PDF vagy Mentés másként → PowerPoint-vetítés (.ppsx).

Szakkifejezések

Fogalom	Magyarázat	Fogalom	Magyarázat
Diaminta	A bemutató mestersablonja: az egységes elrendezések és formázások egy helyen.	Elrendezés	A dia tartalmi helyőrzőinek elosztása (cím, tartalom, kép).
Téma	Összehangolt színek, betűtípusok és effektek.	Helyőrző	Az elrendezés kijelölt helye szövegnek, képnek, táblázatnak.
Átmenet	A diák közötti váltás effektje.	Animáció	Objektumra beállított mozgás: belépés, kiemelés, kilépés, mozgásút.
SmartArt	Szövegből készülő, szerkeszthető ábra (lista, folyamat, hierarchia).	Előadói nézet	Csak az előadónak látszó nézet: jegyzet, következő dia, időmérő.
Szakasz	Diák csoportosítása, hogy könnyebb legyen szervezni a bemutatót.	Képarány	A dia szélesség–magasság aránya (16:9, 4:3).
Hiperhivatkozás	Ugrás másik diára, fájlra vagy weboldalra.	Kontraszt	A szöveg és a háttér különbsége, az olvashatóság alapja.

Biztonsági kérdések – rövid mintaválaszok

Kérdés	Rövid válasz
Mi a különbség az átmenet és az animáció között?	Az átmenet a diák közti váltás, az animáció egy dián belül mozgat objektumot.
Miért használunk diamintát?	Egy helyen módosítható az arculat, minden dia egységes lesz, és új dia is automatikusan követi.
Hogyan lesz olvasható egy dia?	Kevés szöveg, nagy betű, erős kontraszt, ábra és kép, legfeljebb két betűtípus.
Hogyan adod át a bemutatót, ha nincs PowerPoint a gépen?	PDF-ként, vagy .ppsx vetítésként (megnyitáskor automatikusan indul), vagy videóként exportálva.
Mire jók a jegyzetek?	Az előadó emlékeztetője, vetítéskor nem látszik a közönségnek; kinyomtatható jegyzetoldalként is.
Mikor haszontalan az animáció?	Ha elvonja a figyelmet; csak a tartalmat segítő mozgás (pl. lépésenkénti megjelenítés) indokolt.

Gyakori hibák

- Túl sok szöveg a dián, a diát felolvassák.
- Sok, véletlenszerű effekt, különböző átmenetek minden dián.
- Kis kontraszt vagy túl kicsi betű; nem egységes betűtípusok és színek.
- A diaszám, forrás vagy cím hiánya; a diamintától eltérő kézi formázás.
- A bemutató nem fut végig a gépen: előbb próbáld ki vetítésben.

5. Publikálás a világhálón (HTML5, CSS)

Webes alapfogalmak · HTML szerkezet · elemek · CSS · reszponzív oldal · akadálymentesség

A mondandó gerince (tételvázlat, ellenőrzőlista)

- **Alapfogalmak:** internet és web (WWW), kliens–szerver, böngésző, **URL** (protokoll://domain/útvonalt?paraméter#horgony), DNS, **HTTP / HTTPS** (titkosítás), tárhely, domain, feltöltés FTP/SFTP-vel, statikus és dinamikus oldal, CMS (pl. WordPress).
- **Munkamegosztás:** a **HTML** a szerkezetet és a tartalmat adja, a **CSS** a megjelenést, a **JavaScript** a viselkedést.
- **HTML dokumentum szerkezete:** <!DOCTYPE html>, <html lang=„hu”>, <head> (meta charset UTF-8, title, stíluslap, viewport), <body>.
- **Elemek:** címsorok (h1–h6), p, br, hr, strong/em, a (href, relatív és abszolút útvonalt), **img** (src, alt), listák (ul, ol, li), **táblázat** (table, thead, tbody, tr, th, td, colspan, rowspan), űrlap (form, input, label, select, textarea, button), div és span; **szemantikus elemek:** header, nav, main, section, article, aside, footer.
- **CSS:** szelektorok (elem, **.osztály**, **#azonosító**, leszármazott, gyerek, :hover), tulajdonságok (color, background, font, text-align, margin, padding, border, width, display), **doboz-modell** (tartalom, padding, border, margin), kaszkád és specifitás, öröklődés.
- **CSS bekötése:** külső (link rel=stylesheet, ez az ajánlott), belső (style elem), soron belüli (style attribútum). Egységek: px, em, rem, %, vw; színek: név, #hex, rgb(), hsl().
- **Reszponzív oldal:** viewport meta + rugalmas egységek + media query (@media), flexbox / grid.
- **Akadálymentesség és SEO:** alt szöveg, szemantikus szerkezet, kontraszt, értelmes címsor-hierarchia, title és meta description.
- **Gyakorlat:** UTF-8 kódolás, kisbetűs ékezet nélküli fájlnévek, mappaszerkezet (index.html, css/, img/), validálás (W3C), böngésző fejlesztői eszközei (F12). Jogi oldal: szerzői jog, adatvédelem, sütik.

HTML-váz (gyorsan begépelhető)

```
<!DOCTYPE html>
<html lang="hu">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Oldalcím</title>
  <link rel="stylesheet" href="stilus.css">
</head>
<body>
  <header><h1>Főcím</h1>
    <nav><a href="index.html">Kezdőlap</a> <a href="kapcsolat.html">Kapcsolat</a></nav>
  </header>
  <main>
    <section>
      <h2>Szakasz</h2>
      <p>Szöveg <strong>kiemelve</strong>.</p>
      
      <ul><li>Egy</li><li>Kettő</li></ul>
    </section>
  </main>
  <footer>&copy; 2026</footer>
</body>
</html>
```

CSS-váz (stilus.css)

```
body { font-family: Arial, sans-serif; margin: 0; }
header { background: #1F5FA8; color: white; padding: 1rem; }
nav a { color: white; margin-right: 1rem; text-decoration: none; }
nav a:hover { text-decoration: underline; }
table { border-collapse: collapse; }
th, td { border: 1px solid #999; padding: 4px 8px; }
tr:nth-child(even) { background: #f0f0f0; }
.kiemelt { font-weight: bold; color: #c0392b; }
#fo { max-width: 800px; margin: auto; }
@media (max-width: 600px) { nav a { display: block; } }
```

Gépes bemutatók (kérésre, lépésről lépésre)

Bemutató	Lépések
Oldalváz gyorsan	VS Code vagy Notepad++ → új fájl → az alábbi HTML-váz begépelése (ez 3 perc) → mentés UTF-8-ban index.html néven → megnyitás böngészőben.
Külső stíluslap	Új fájl stilus.css → az oldal <head> részében <link rel=„stylesheet” href=„stilus.css”> → módosítás és frissítés (F5). Mutasd meg a kaszkádot : az osztály felülírja az elem szelektort.
Táblázat formázása	table, thead, tbody, th, td; CSS: border-collapse: collapse; th, td { border ... }; tr:nth-child(even) { background ... } (sávos sorok).
Navigációs menü	nav > a elemek; CSS: display: flex vagy inline; :hover állapot; relatív linkek a többi oldalra.
Reszponzív változat	<meta name=„viewport” ...> és @media (max-width: 600px) { ... }; a böngészőben szűkítsd az ablakot (F12 → eszköz-nézet) és mutasd meg a változást.
Űrlap	form, label, input (text, email, number, checkbox, radio), select, textarea, button; method és action; required attribútum.

Szakkifejezések

Fogalom	Magyarázat	Fogalom	Magyarázat
URL	A web-erőforrás egyedi címe (protokoll, domain, útvonal).	DNS	A domainnevet IP-címre fordító rendszer.
HTTP / HTTPS	A weboldalak átviteli protokollja; a HTTPS titkosított.	Szemantikus elem	Jelentést hordozó elem (header, nav, main, footer).
Szelektor	A CSS-nek megmondja, mely elemekre vonatkozzon a szabály.	Doboz-modell	Tartalom + padding + border + margin.
Kaskád	A szabályok felülírási sorrendje (specifititás, sorrend).	Relatív útvonal	Az aktuális fájlhoz képest megadott hivatkozás (img/kep.jpg).
Reszponzív	A megjelenés alkalmazkodik a kijelző méretéhez.	alt attribútum	A kép szöveges megfelelője (képernyőolvasó, hibás kép).
CMS	Tartalomkezelő rendszer, kód nélküli szerkesztéshez.	Validálás	A kód szabványoknak való megfelelésének ellenőrzése.

Bizottsági kérdések – rövid mintaválaszok

Kérdés	Rövid válasz
Mi a különbség a HTML és a CSS között?	A HTML a tartalmat és a szerkezetet írja le, a CSS a megjelenést (szín, méret, elhelyezés).
Osztály és azonosító különbsége?	Az osztály (.nev) több elemre alkalmazható, az azonosító (#nev) egyedi, egy oldalon csak egyszer szerepelhet.
Mi a doboz-modell?	Minden elem egy doboz: tartalom, körülrötte padding, border, kívül margin.
Miért kell az alt attribútum?	Képernyőolvasónak, hibás képnél és keresőknek is megmutatja a kép tartalmát: akadálymentességi és SEO szempont.
Relatív és abszolút útvonal?	Relatív: az aktuális oldalhoz képest (img/a.jpg). Abszolút: teljes URL (https://...).
Mitől reszponzív egy oldal?	Viewport meta, rugalmas egységek (% , rem) és media query-k, amik a kijelző szélességétől függően változtatják a stílust.

Gyakori hibák

- Hiányzó <meta charset=„UTF-8”>: az ékezetes betűk elromlanak.
- Táblázat elrendezésre használata: a táblázat adatokhoz való, az elrendezést CSS (flex, grid) végzi.
- Stílus soron belül mindenhol: nehéz karbantartani, használj külső CSS-t.
- Ékezetes, szóközös fájlnev és abszolút (C:\...) útvonal: máshol nem működik.
- alt nélküli kép, rosszul záródó elemek, nem létező mappa a hivatkozásban.

6. Táblázatkezelés (Excel 2021)

Hivatkozások · függvények · adatkezelés · kimutatás · diagram · feltételes formázás

A mondandó gerince (tételvázlat, ellenőrzőlista)

- Felépítés:** munkafüzet, munkalap, cella, cellacím (A1), tartomány, sor/oszlop. **Adattípusok:** szám, szöveg, dátum/idő (sorszámként tárolva), logikai, hibaérték. A **cellaformátum nem az érték:** csak a megjelenést adja.
- Képletek:** operátorok és kiértékelési sorrend, **hivatkozások:** relatív (A1), abszolút (\$A\$1), vegyes (A\$1, \$A1), más munkalapra (Munka2!A1), névvel ellátott tartomány; a másolás hatása a hivatkozásra.
- Függvénycsoportok:** logikai (HA, ÉS, VAGY), matematikai (SZUM, KEREKÍTÉS, MARADÉK), statisztikai (ÁTLAG, DARAB, DARABTELI, SZUMHATÖBB), keresési (XKERES, FKERES, INDEX+HOL.VAN), szöveges (BAL, KÖZÉP, SZÖVEG.KERES), dátum (ÉV, HÓNAP, HÉT.NAPJA). A teljes lista az írásbeli segédletben van.
- Excel 2021 újdonságai:** dinamikus tömbök (SZŰRŐ, EGYEDI, SORBA.RENDEZ) és XKERES. Az eredmény „kiömlik” a szomszédos cellákba; hivatkozás a kiömlött tartományra: A2#.
- Adatkezelés:** rendezés (többszintű), szűrés (automatikus, irányított), Excel-táblázat (Ctrl+T), részösszeg, ismétlődések eltávolítása, **adatérvényesítés** (legördülő lista), szövegből oszlopok, külső adat (CSV) importálása.
- Kimutatás:** nagy adattáblából összegző táblát készít (sor, oszlop, érték, szűrő mezők; összeg, darab, átlag; csoportosítás); frissíteni kell, ha az adat változik; kimutatás-diagram.
- Diagramok:** típusok (oszlop, vonal, kör, pont/XY, terület, kombinált), elemei (cím, tengelyek, jelmagyarázat, adatfeliratok, rácsvonal, trendvonal). Melyik mire: összehasonlítás → oszlop, időbeli változás → vonal, arány → kör, összefüggés → pont. Kerül a félrevezető skálát.
- Formázás:** számformátumok, **feltételes formázás** (adatsáv, színskála, képlet alapú szabály), cellastílus, szegély, oldalbeállítás, nyomtatási terület, ismétlődő címsorok.
- Védelem és együttműködés:** cella zárolása és lapvédelem, megjegyzések, védett munkafüzet.

Gépes bemutatók (kérésre, lépésről lépésre)

Bemutató	Lépések
Hivatkozások (szorzótábla)	B1:K1 és A2:A11 számokkal. A B2 cella: =A2*B\$1 , majd másolás jobbra és lefelé. Mutasd meg, melyik hivatkozás miatt rögzített (F4).
XKERES	=XKERES(A2;\$H\$2:\$H\$20;\$I\$2:\$I\$20;„nincs”), sávós kereséshez: =XKERES(B2;\$K\$2:\$K\$6;\$L\$2:\$L\$6;;-1) (pontos vagy következő kisebb).
SZUMHATÖBB, DARABHATÖBB	=SZUMHATÖBB(\$D\$2:\$D\$100;\$B\$2:\$B\$100;F2;\$C\$2:\$C\$100;;>”&G2). Mutasd meg, hogy az F2 megváltoztatásával az eredmény frissül.
Kimutatás és diagram	Az adat kijelölése → Beszúrás → Kimutatás → mezők húzása (sor: kategória, érték: összeg) → Kimutatáselemzés → Kimutatásdiagram (tengelycímkék, cím).
Feltételes formázás képlettel	Kijelölés (A2:D100) → Kezdőlap → Feltételes formázás → Új szabály → Képlet: =D2>ÁTLAG(\$D\$2:\$D\$100) → formátum. A \$ jelek nélkül elcsúszik!
Adatérvényesítés + HA	Adatok → Adatérvényesítés → Lista (forrás: tartomány) → a mellette lévő cellában HA vagy XKERES a választott érték alapján.
Dinamikus tömb	=SZŰRŐ(A2:D100;C2:C100>50;„nincs”) egy cellában → a lista kiömlik. =SORBA.RENDEZ(EGYEDI(B2:B100)) a kategóriák rendezett listájához.

Szakkifejezések

Fogalom	Magyarázat	Fogalom	Magyarázat
Relatív hivatkozás	Másoláskor a cél helyéhez igazodik (A1 → A2).	Abszolút hivatkozás	Másoláskor nem változik (\$A\$1, F4).
Tartomány	Összefüggő cellák (A1:C10).	Függvény	Előre megírt képlet, névvel és argumentumokkal.
Kimutatás	Összegző tábla, amelyet az adattábla mezőinek húzásával állítunk össze.	Adatérvényesítés	Szabály arra, hogy mit lehet egy cellába beírni (pl. lista).
Feltételes formázás	A formátum az érték vagy képlet alapján automatikusan változik.	Névvel ellátott tartomány	Egy tartomány szöveges neve, amire hivatkozhatunk.
Excel-táblázat	Strukturált, bővíthető adatterület (Ctrl+T) szűrővel és oszlopnevekkel.	Trendvonal	A diagram adatsorához illesztett közelítő vonal.
Kiömlés	A dinamikus tömbképlet eredménye több cellába terül szét.	Kerekítés	Az érték módosítása (KEREKÍTÉS), nem csak a megjelenítés.

Bizottsági kérdések – rövid mintaválaszok

Kérdés	Rövid válasz
Relatív és abszolút hivatkozás?	A relatív másoláskor eltolódik, az abszolút (\$A\$1) nem. A vegyes csak az oszlopot vagy a sort rögzíti.
FKERES vagy XKERES?	Az XKERES bármely irányba keres, nem függ az oszlopsorrendtől, van „ha nincs” argumentuma, és pontos egyezés az alapértelmezett.
Mikor használj kimutatást?	Nagy adattábla összegzésére és csoportosítására képletek nélkül, gyors átrendezéssel.
Szűrés és rendezés különbsége?	A rendezés átrendezi a sorokat, a szűrés elrejt azokat, amelyek nem felelnek meg a feltételnek.
Melyik diagramtípus mire jó?	Oszlop: összehasonlítás, vonal: időbeli alakulás, kör: részarány, pont: két mennyiség összefüggése.
Mi az adatérvényesítés?	Korlátozza vagy listából kínálja a megadható értékeket (pl. lenyíló menü), így kevesebb a hibás adat.

Gyakori hibák

- Szöveggént tárolt szám (pl. importálás után): a SZUM nem számolja, a keresés nem találja.
- Egyesített cellák az adattáblában: a rendezés és a szűrés elromlik.
- Kézzel beírt érték képlet helyett, vagy hiányzó \$ jelek, ezért elcsúszik a tartomány.
- Csonkolt tengelyű vagy 3D diagram, ami félrevezet; hiányzó cím és tengelyfelirat.
- A kimutatás nem frissül az adat módosítása után.

7. Adatbázis-kezelés (MariaDB 10 + phpMyAdmin 5.2)

Relációs modell · kulcsok · normalizálás · SQL · phpMyAdmin

A mondandó gerince (tételvázlat, ellenőrzőlista)

- **Alapfogalmak:** adat és információ, adatbázis, **adatbázis-kezelő rendszer (DBMS)**. **Relációs modell:** tábla (reláció), rekord (sor), mező (oszlop). **Kulcsok:** elsődleges (PK, egyedi azonosító), idegen (FK, másik tábla PK-jára mutat), összetett kulcs.
- **Adatmodellezés (ER-modell):** egyed, tulajdonság, kapcsolat; kapcsolattípusok **1:1**, **1:N**, **N:M**. Az N:M kapcsolatot **kapcsolótáblával** oldjuk fel (két idegen kulcs).
- **Redundancia és anomáliák:** ismétlődő adat → módosítási, beszúrási, törlési anomália. **Normalizálás:** 1NF (atomi mezők), 2NF (nincs részleges függés az összetett kulcstól), 3NF (nincs tranzitív függés).
- **Integritás:** entitás- (PK egyedi, nem NULL), **hivatkozási** (FK létező PK-ra mutat; ON DELETE / ON UPDATE), adattípus-megkötések (NOT NULL, UNIQUE, DEFAULT, CHECK).
- **Adattípusok:** INT, DECIMAL(p,s), FLOAT, VARCHAR(n), CHAR(n), TEXT, DATE, TIME, DATETIME, BOOLEAN / TINYINT(1).
- **SQL nyelvcsoportok:** **DDL** (CREATE, ALTER, DROP), **DML** (INSERT, UPDATE, DELETE), **DQL** (SELECT), **DCL** (GRANT, REVOKE), **TCL** (COMMIT, ROLLBACK). **SELECT:** WHERE, GROUP BY, HAVING, ORDER BY, LIMIT; aggregáló függvények; JOIN-ok (INNER, LEFT, RIGHT); allekérdezés; **nézet (VIEW)**; index.
- **phpMyAdmin:** adatbázis létrehozása, tábla létrehozása (név, mezők, típus, **A_I** = AUTO_INCREMENT, PK), Struktúra fül, **Tervező** (kapcsolatok), SQL fül, Importálás / Exportálás (.sql, .csv).

Példa: táblák kapcsolattal és egyösszesítő lekérdezés

```
CREATE TABLE osztaly (id INT AUTO_INCREMENT PRIMARY KEY, nev VARCHAR(10) NOT NULL);
CREATE TABLE tanulo (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nev VARCHAR(50) NOT NULL,
  osztaly_id INT,
  FOREIGN KEY (osztaly_id) REFERENCES osztaly(id)
);
SELECT o.nev AS osztaly, COUNT(*) AS letszam
FROM tanulo t INNER JOIN osztaly o ON t.osztaly_id = o.id
GROUP BY o.nev
HAVING COUNT(*) > 20
ORDER BY letszam DESC;
```

Normalizálás példa: a rendeles(id, vevo_nev, vevo_cim, termék_nev, termék_ar) ismétli a vevő és a termék adatait → bontsd **vevo**, **termék**, **rendeles** táblákra, idegen kulcsokkal.

Gépes bemutatók (kérésre, lépésről lépésre)

Bemutató	Lépések
Táblák és kapcsolat SQL-lel	SQL fül: CREATE TABLE osztaly (...); CREATE TABLE tanulo (... FOREIGN KEY (osztaly_id) REFERENCES osztaly(id)); mutasd meg a Struktúra és a Tervező nézetet.
Adatkezelés	INSERT INTO ... VALUES (...); UPDATE ... SET ... WHERE ...; DELETE FROM ... WHERE ...; mutasd meg a WHERE hiányának veszélyét.
Lekérdezés JOIN-nal	SELECT ... FROM tanulo t INNER JOIN osztaly o ON t.osztaly_id = o.id GROUP BY ... HAVING ... ORDER BY ... (lásd lent és az SQL-segédletet).
Tervező nézet	Tábla → Több → Tervező (Designer): idegen kulcs-kapcsolat húzással vagy a Struktúra → Kapcsolatok nézet. Magyarázd el a hivatkozási integritást.

Szakkifejezések

Fogalom	Magyarázat	Fogalom	Magyarázat
Elsődleges kulcs (PK)	A rekordot egyértelműen azonosító mező vagy mezők.	Idegen kulcs (FK)	Másik tábla PK-jára hivatkozó mező, ez teremti a kapcsolatot.
Redundancia	Az azonos adat felesleges ismétlődése.	Anomália	Hiba, amit a redundancia okoz módosításkor, beszúráskor, törléskor.
Normálforma	A táblaszerkezet minőségi szintje (1NF, 2NF, 3NF).	JOIN	Táblák összekapcsolása közös érték alapján.
Aggregáció	Csoportonkénti összesítés (COUNT, SUM, AVG, MIN, MAX).	Nézet	Mentett lekérdezés, ami virtuális táblaként viselkedik.
Hivatkozási integritás	Az idegen kulcsnak mindig létező elsődleges kulcsra kell mutatnia.		

Bizottsági kérdések – rövid mintaválaszok

Kérdés	Rövid válasz
Mi a különbség a PK és az FK között?	A PK egy tábla rekordjait azonosítja, az FK egy másik tábla PK-ját hivatkozza, és kapcsolatot hoz létre.
Miért normalizálunk?	A redundancia és az anomáliák csökkentése miatt: kevesebb duplikált adat, biztonságosabb módosítás.
INNER JOIN és LEFT JOIN különbsége?	Az INNER JOIN csak a párosítható sorokat adja, a LEFT JOIN a bal tábla összes sorát megtartja (NULL a hiányzó párnál).
WHERE és HAVING?	A WHERE a csoportosítás előtt szűri a sorokat, a HAVING a csoportosítás után a csoportokat (aggregátumra).
Hogyan oldasz fel egy N:M kapcsolatot?	Kapcsolótáblával, amely két idegen kulcsot tartalmaz a két táblára.

Gyakori hibák

- Nincs elsődleges kulcs; több érték egy mezőben; dátum vagy szám szöveggént tárolva (rosszul rendez, nehéz számolni vele).
- JOIN feltétel nélkül (descartes-szorzat); UPDATE vagy DELETE WHERE nélkül (minden sor érintett).

8. B) Programozás – módszer és programozási tételek

Kódolás · kódmódosítás · kódértelmezés · pszeudokód ↔ C++ · szóbeli magyarázat

Mit várnak a B) feladtnál?

- Három feladattípus:** (1) **kódolás:** a leírt feladathoz programot írsz, (2) **kódmódosítás:** egy adott programot átírsz, (3) **kódértelmezés:** pszeudokódról vagy kódról megmondod, mit csinál. A B) feladat megoldása a **felkészülési időbe** esik, a felelet alatt bemutatod.
- A legtöbb feladat egy vagy több **programozási tételre** vezethető vissza (összegzés, megszámlálás, maximum, eldöntés, keresés, kiválogatás, szétválogatás, rendezés, metszet/unió). Ha azonosítod a tételt, a program fele megvan.
- A bizottság azt akarja látni, hogy **érted**, amit írtál: ezért mindig mondd el az ötletet, és készülj módosításra („mi változik, ha a legkisebbet kérjük?”).

Hat lépés a megoldáshoz (és a felelethez)

Lépés	Mit csinálsz	Mit mondasz a felelet alatt
1. Specifikáció	Bemenet, kimenet, megkötések (üres bemenet? negatív szám?).	„A feladat: ..., a bemenet ..., a kimenet ...”
2. Példa kézzel	Egy kis, ellenőrizhető adat, amelynek ismered a megoldását.	„Például ennél a bemenetnél az eredménynek ... kell lennie.”
3. Tétel azonosítása	Megszámlálás? Maximum? Kiválogatás? Rendezés?	„Ez a ... programozási tétel.”
4. Terv	Pszeudokód vagy 2–3 mondatos terv; kezdőérték és ciklus.	„A kezdőérték ..., a ciklus végigmegy ..., és ha ..., akkor ...”
5. Kód és teszt	Programozd le, majd teszt: alap, szélső eset (üres, egy elem), azonos értékek .	Futtasd, és mutasd meg a tesztek.
6. Módosításra készülj	Feltétel, reláció ($> \leftrightarrow \geq$), rendezés iránya, minimum maximum helyett.	„Ha a feltétel ..., akkor itt ezt a sort írnám át.”

Magyarázat-sablon (kb. 1 perc): „A feladat A bemenet ..., a kimenet Ez a [tétel neve] programozási tétel: a kezdőértéket [...] -ra állítom, végigmegyek az elemeken, és ha [feltétel], akkor [teendő]. Üres bemenetnél [viselkedés]. Kipróbáltam [2–3 adattal]. Ha a feladat [módosítás] lenne, akkor [melyik sor] változna.”

Pszeudokód és C++ megfelelője

Pszeudokód	C++	Figyelj rá
$x := 5$	<code>int x = 5;</code>	Az értékadás = , az összehasonlítás ==.
Ha feltétel akkor ... különben ... Elágazás vége	<code>if (feltétel) { ... } else { ... }</code>	A feltételhez nem kell 'akkor', de kell zárójel.
Ciklus $i := 1$ -től N -ig ... Ciklus vége	<code>for (int i = 0; i < n; i++) { ... }</code>	A pszeudokód tömbje 1-től , a C++ 0-tól indexel!
Ciklus amíg feltétel ... Ciklus vége	<code>while (feltétel) { ... }</code>	A feltétel igaz esetén megy tovább.
Ciklus ... amíg feltétel (hátultesztelő)	<code>do { ... } while (!feltétel);</code>	A hátultesztelő legalább egyszer lefut; a feltétel megfordul.
$a \text{ div } b, a \text{ mod } b$	<code>a / b</code> (egészekenél), <code>a % b</code>	Egész osztás: a / b csak egész típusoknál, tizedeshez (double) kell.
Be: x / Ki: x	<code>cin >> x;</code> / <code>cout << x;</code>	A pszeudokódban a kiírás szövegei lehetnek eltérőek.
Függvény $F(T, N)$: Egész	<code>int f(const vector<int>& t)</code>	A vector tudja a méretét: <code>t.size()</code> .
Csere(a, b)	<code>swap(a, b);</code>	Az algorithm fejléc kell hozzá.

A hivatalos érettségi pszeudokód jelölése kissé eltérhet az itt használttól, de a logika ugyanaz: kezdőérték, ciklus, feltétel. Ne a jelölés, hanem a **folyamat** megértése számít.

Programozási tételek – pszeudokód, C++, magyarázat

A C++ kódok `vector<int>` adatra készültek (0-tól indexelve). Szükséges fejlécek: `iostream`, `vector`, `string`, `sstream`, `fstream`, `ioomanip`, `cmath`, `algorithm`, `map`, `cctype`, és a `using namespace std;` sort.

1. Sorozatszámítás (összegzés)

Pszeudokód	C++
$s := 0$ Ciklus $i := 1$ -től N -ig $s := s + T[i]$ Ciklus vége	<pre>int osszeg(const vector<int>& t) { int s = 0; for (int x : t) s += x; return s; }</pre> // üres sorozatnál az eredmény 0

Szóban: Egy sorozatból egyetlen értéket számolunk (összeg, szorzat). A kezdőérték a **művelet egységeleme**: összegnél 0, szorzatnál 1.

Tipikus módosítás: szorzat (kezdőérték 1), átlag ((double) osztás, üres sorozat védelme), csak a páros elemek összege, mozgóátlag.

2. Megszámlálás

Pszeudokód	C++
$db := 0$ Ciklus $i := 1$ -től N -ig Ha $T[i] \bmod 2 = 0$ akkor $db := db + 1$ Elágazás vége Ciklus vége	<pre>int paros_db(const vector<int>& t) { int db = 0; for (int x : t) if (x % 2 == 0) db++; return db; }</pre> // a feltétel cserélhető

Szóban: Megszámlaljuk, hány elem teljesít egy feltételt: a számlálót 0-ról indul, és minden megfelelő elemnél eggyel nő.

Tipikus módosítás: másik feltétel ($x > 10, x \% 3 == 0$), két feltétel együtt ($\&\&$), a darabszám százalékos aránya.

3. Maximumkiválasztás (a hely)

Pszudokód	C++
<pre>mi := 1 Ciklus i := 2-től N-ig Ha T[i] > T[mi] akkor mi := i Elágazás vége Ciklus vége Max := T[mi]</pre>	<pre>int max_index(const vector<int>& t) { // nem üres vektorra! int mi = 0; for (int i = 1; i < (int)t.size(); i++) if (t[i] > t[mi]) mi = i; // >: első, >=: utolsó maximum return mi; }</pre>

Szóban: A legnagyobb elem **helyét** tartjuk nyilván: az elsőt tekintjük legnagyobbknak, a többit hozzá hasonlítjuk. A helyből az érték is elérhető. Az üres sorozat külön eset.

Tipikus módosítás: minimum (reláció megfordítva), > és >= különbsége (első vagy utolsó maximum), az összes maximum kiírása, a maximum előfordulásainak száma.

4. Eldöntés

Pszudokód	C++
<pre>i := 1 Ciklus amíg i <= N és T[i] >= 0 i := i + 1 Ciklus vége Van := (i <= N)</pre>	<pre>bool van_negativ(const vector<int>& t) { int i = 0; while (i < (int)t.size() && t[i] >= 0) i++; // amíg nincs ilyen elem return i < (int)t.size(); }</pre>

Szóban: Eldöntjük, van-e adott tulajdonságú elem. Az **első** találatnál megállunk, ezért while ciklust használunk, nem végigmegyünk feleslegesen.

Tipikus módosítás: „minden elemre igaz-e?” (a tagadott feltételre keresünk ellenpéldát), másik feltétel, két szomszédos azonos elem keresése.

5. Keresés (lineáris)

Pszudokód	C++
<pre>i := 1 Ciklus amíg i <= N és T[i] ≠ x i := i + 1 Ciklus vége Ha i <= N akkor Hely := i különben Hely := 0</pre>	<pre>int keres(const vector<int>& t, int x) { // az első találat indexe, vagy -1 int i = 0; while (i < (int)t.size() && t[i] != x) i++; return i < (int)t.size() ? i : -1; }</pre>

Szóban: Az elejétől haladunk, az első találatnál megállunk, és visszaadjuk a helyét; ha nincs találat, -1-et adunk. Rendezetlen sorozatban ez a legegyszerűbb módszer.

Tipikus módosítás: az utolsó előfordulás (hátról haladva), az összes előfordulás helye, keresés feltétel alapján (nem egyenlőség).

6. Kiválogatás

Pszudokód	C++
<pre>db := 0 Ciklus i := 1-től N-ig Ha T[i] mod 2 = 0 akkor db := db + 1 R[db] := T[i] Elágazás vége Ciklus vége</pre>	<pre>vector<int> parosak(const vector<int>& t) { vector<int> r; for (int x : t) if (x % 2 == 0) r.push_back(x); return r; }</pre>

Szóban: A feltételnek megfelelő elemeket új sorozatba másoljuk. A vector push_back-je miatt nem kell előre tudnunk, hány elem lesz.

Tipikus módosítás: másik feltétel, az indexek kiválogatása az értékek helyett, kiválogatás szöveges adatból.

7. Szétválogatás

Pszudokód	C++
<pre>Ciklus i := 1-től N-ig Ha T[i] > 0 akkor P-be tesz T[i] különben ha T[i] < 0 akkor Neg-be tesz T[i] különben Nulla-ba tesz T[i] Elágazás vége Ciklus vége</pre>	<pre>void szetvalogat(const vector<int>& t, vector<int>& poz, vector<int>& neg, vector<int>& nulla) { for (int x : t) { if (x > 0) poz.push_back(x); else if (x < 0) neg.push_back(x); else nulla.push_back(x); } }</pre>

Szóban: Egy sorozatot egy menetben szétosztunk két vagy több sorozatba a feltétel szerint. Minden elem pontosan egy helyre kerül.

Tipikus módosítás: páros/páratlan szétosztás, két csoport helyett három, a csoportok átlagának kiszámítása.

8a. Rendezés: minimumkiválasztásos

Pszudokód	C++
<pre>Ciklus i := 1-től N-1-ig mi := i Ciklus j := i+1-től N-ig Ha T[j] < T[mi] akkor mi := j Elágazás vége Ciklus vége Csere(T[i], T[mi]) Ciklus vége</pre>	<pre>void rendez_min(vector<int>& t) { // minimumkiválasztásos rendezés int n = t.size(); for (int i = 0; i < n - 1; i++) { int mi = i; for (int j = i + 1; j < n; j++) if (t[j] < t[mi]) mi = j; swap(t[i], t[mi]); } }</pre>

Szóban: Az i. helyre mindig a maradék sorozat legkisebb elemét tesszük: megkeressük a legkisebb elem helyét, és kicseréljük az i. elemmel.

Tipikus módosítás: csökkenő sorrend (< helyett >), stringek rendezése, rendezés egy struktúra mezője szerint. Az STL-ben: sort(t.begin(), t.end()).

8b. Rendezés: buborékredezés

Pszudokód	C++
<pre>Ciklus i := N-től 2-ig (lefelé) Ciklus j := 1-től i-1-ig Ha T[j] > T[j+1] akkor Csere(T[j], T[j+1]) Elágazás vége Ciklus vége Ciklus vége</pre>	<pre>void buborek(vector<int>& t) { // buborékredezés int n = t.size(); for (int i = n - 1; i > 0; i--) for (int j = 0; j < i; j++) if (t[j] > t[j + 1]) swap(t[j], t[j + 1]); // csökkenőhöz: < }</pre>

Szóban: A szomszédos elemeket hasonlítjuk, és ha rossz a sorrend, felcseréljük: minden menetben a maradék legnagyobb elem a helyére „úszik”, mint egy buborék.

Tipikus módosítás: csökkenő sorrend (> helyett <), korai leállás, ha egy menetben nem történt csere. Mindkét rendezés lépésszáma kb. N^2 , az STL sort ennél gyorsabb.

9. Metszet és unió (rendezett sorozatokra)

Pszudokód	C++
<pre>i := 1; j := 1 Ciklus amíg i <= N és j <= M Ha A[i] = B[j] akkor R-be tesz A[i]; i++; j++ különben ha A[i] < B[j] akkor i := i + 1 különben j := j + 1 Elágazás vége Ciklus vége (unió: a kisebbet tesszük R-be, majd a maradékot hozzáfűzzük)</pre>	<pre>vector<int> metszet(const vector<int>& a, const vector<int>& b) { // rendezett bemenetek vector<int> r; int i = 0, j = 0; while (i < (int)a.size() && j < (int)b.size()) { if (a[i] == b[j]) { r.push_back(a[i]); i++; j++; } else if (a[i] < b[j]) i++; else j++; } return r; } vector<int> unio(const vector<int>& a, const vector<int>& b) { vector<int> r; int i = 0, j = 0; while (i < (int)a.size() && j < (int)b.size()) { if (a[i] == b[j]) { r.push_back(a[i]); i++; j++; } else if (a[i] < b[j]) r.push_back(a[i++]); else r.push_back(b[j++]); } while (i < (int)a.size()) r.push_back(a[i++]); // a maradék elemek while (j < (int)b.size()) r.push_back(b[j++]); return r; }</pre>

Szóban: Két rendezett sorozatot két indexszel egyszerre járunk be („összefuttatás”): metszetenél csak az egyenlőket vesszük, uniónál a kisebbiket másoljuk, a végén a maradékot hozzáfűzzük.

Tipikus módosítás: unió helyett különbség ($A \setminus B$), ismétlődő elemek kezelése, rendezetlen bemenet (előbb rendezni kell).

10. Bináris keresés (rendezett sorozatban)

Pszudokód	C++
<pre>a := 1; f := N Ciklus amíg a <= f k := (a + f) div 2 Ha T[k] = x akkor Hely := k; vége különben ha T[k] < x akkor a := k+1 különben f := k-1 Ciklus vége</pre>	<pre>int binaris_keres(const vector<int>& t, int x) { // csak rendezett t-re int a = 0, f = (int)t.size() - 1; while (a <= f) { int k = (a + f) / 2; if (t[k] == x) return k; if (t[k] < x) a = k + 1; else f = k - 1; } return -1; }</pre>

Szóban: Rendezett sorozatban mindig a középső elemet nézzük, és a keresett érték felé a fél tartományt eldobjuk. Lépésszáma kb. $\log_2 N$, ezért nagy adatnál sokkal gyorsabb a lineárisnál.

Tipikus módosítás: az első vagy az utolsó előfordulás megkeresése, rendezetlen bemenet kezelése, keresés rekordokban egy mező szerint.

Számelméleti és szöveges rutinok (gyorsan megírható)

```
bool prim(int n) {
    if (n < 2) return false;
    for (int i = 2; (long long)i * i <= n; i++)
        if (n % i == 0) return false;
    return true;
}

int lanko(int a, int b) { while (b != 0) { int r = a % b; a = b; b = r; } return a; }    // euklideszi algoritmus
int szamjegyOsszeg(int n) { int s = 0; for (n = abs(n); n > 0; n /= 10) s += n % 10; return s; }
bool szokoev(int ev) { return (ev % 4 == 0 && ev % 100 != 0) || ev % 400 == 0; }
bool palindrom(string s) { string r = s; reverse(r.begin(), r.end()); return s == r; }
```

További, az írásbeli segédletben szereplő rutinok: bináris átváltás, idő formázása (setw/setfill), a rendezés több szempont szerint (lambdával), és az STL-függvények (count_if, max_element, find_if, accumulate).

9. B) Kidolgozott mintafeladatok

Hét kódolási feladat tesztekkel · három kódértelmezési feladat megoldással

Mindegyik kódot lefordítottam és kipróbáltam. A feladatok **saját mintafeladatok**, nem hivatalosak. A program elején szükséges: `#include <iostream>, <vector>, <string>, <sstream>, <fstream>, <iomanip>, <cmath>, <map>, <cctype>` és a `using namespace std;`

M1 – Legnagyobb elem, helye és előfordulása

Feladat: Egy egész számokat tartalmazó sorozatban add meg a legnagyobb elemet, első helyét és azt, hányszor fordul elő.

Ötlet: Maximumkiválasztás + megszámlálás egy menetben. Ha új maximumot találunk, a számlálót újratezdjük (1), ha egyenlőt, növeljük.

```
// Legnagyobb elem, első helye és előfordulási száma
void legnagyobb(const vector<int>& t) {
    int mi = 0, db = 1;
    for (int i = 1; i < (int)t.size(); i++) {
        if (t[i] > t[mi]) { mi = i; db = 1; } // új maximum: a számlálót újratezdjük
        else if (t[i] == t[mi]) db++;
    }
    cout << "Maximum: " << t[mi] << ", első helye: " << mi + 1 << ". elem, " << db << " db" << endl;
}
```

Teszt: {3, -1, 4, 1, -5, 9, 2, 6, 5, 3, 9} → „Maximum: 9, első helye: 6. elem, 2 db”. Szélső eset: egy elemű sorozat.

Módosítás: minimum kérésénél a reláció megfordul; az utolsó előfordulás helyéhez a > helyett >= kell.

M2 – Másodfokú egyenlet

Feladat: Olvass be három számot (a, b, c), és oldd meg az $ax^2 + bx + c = 0$ egyenletet.

Ötlet: Elágazás a diszkrimináns ($D = b^2 - 4ac$) szerint: $D < 0$ nincs valós gyök, $D = 0$ egy gyök, $D > 0$ két gyök. Az $a = 0$ külön eset (nem másodfokú).

```
// Másodfokú egyenlet megoldása
int main() {
    double a, b, c;
    cout << "a, b, c: ";
    cin >> a >> b >> c;
    if (a == 0) { cout << "Nem másodfokú az egyenlet." << endl; return 0; }
    double d = b * b - 4 * a * c; // diszkrimináns
    if (d < 0) cout << "Nincs valós gyök." << endl;
    else if (d == 0) cout << "Egy gyök: " << -b / (2 * a) << endl;
    else cout << "Két gyök: " << (-b + sqrt(d)) / (2 * a) << " és " << (-b - sqrt(d)) / (2 * a) << endl;
    return 0;
}
```

Teszt: 1 -3 2 → „Két gyök: 2 és 1”; 1 2 1 → „Egy gyök: -1”; 1 2 3 → „Nincs valós gyök.”; 0 2 3 → „Nem másodfokú az egyenlet.”

Módosítás: a $D = 0$ összehasonlítás valós számnál kerekítési hibát okozhat, pontosabb: $\text{abs}(D) < 1e-9$. Komplex gyökök kiírása is kérhető.

M3 – Prímek egy intervallumban

Feladat: Írd ki egy megadott intervallum összes prímszámát, és add meg a darabszámukat.

Ötlet: A prímteszt a 2-től $\sqrt{n}$ -ig próbálkozik osztással (az $i \cdot i \leq n$ feltétellel, long long-gal a túlcsordulás ellen); az intervallumon végigmenő ciklus kiválogat és megszámlál.

```
// Prímek egy intervallumban
bool prim(int n) {
    if (n < 2) return false;
    for (int i = 2; (long long)i * i <= n; i++)
        if (n % i == 0) return false;
    return true;
}

void primek(int also, int felso) {
    int db = 0;
    for (int n = also; n <= felso; n++)
        if (prim(n)) { cout << n << " "; db++; }
    cout << endl << "Darab: " << db << endl;
}
```

Teszt: primek(10, 30) → „11 13 17 19 23 29” és „Darab: 6”. Szélső eset: primek(0, 2) → csak a 2.

Módosítás: az első N prím kiírása, ikerprímek, egy szám prímtenyezős felbontása.

M4 – Szavak száma és a leghosszabb szó

Feladat: Egy beolvasott mondatban számold meg a szavakat, és add meg a leghosszabbat.

Ötlet: A `stringstream >>` operátora szóközönként tördel. A leghosszabb szó maximumkiválasztás a szóhosszra.

```
// Szavak száma és a leghosszabb szó
void szavak(const string& mondat) {
    stringstream ss(mondat);
    string szo, leghosszabb;
    int db = 0;
    while (ss >> szo) {
        db++;
        if (szo.size() > leghosszabb.size()) leghosszabb = szo;
    }
    cout << "Szavak száma: " << db << endl << "Leghosszabb szó: " << leghosszabb << endl;
}
```

Teszt: „A kis kutya gyorsan szaladt” → „Szavak száma: 5”, „Leghosszabb szó: gyorsan”.

Módosítás: Figyelem: UTF-8 kódolásban az ékezetes betű 2 bájtos lehet, ezért a `size()` ékezetes szónál pontatlan hosszt adhat. Ezt említsd meg a bizottságnak.

M5 – Számjegyek megfordítása, palindrom szám

Feladat: Fordítsd meg egy pozitív egész szám számjegyeit, és dönts el, palindrom-e a szám.

Ötlet: A számjegyeket hátulról vesszük ($n \% 10$), az eredmény elé fűzzük ($x \cdot 10 + \text{számjegy}$), majd levágjuk ($n / 10$).

```
// Számjegyek megfordítása, palindrom szám
int megfordit(int n) {
    int x = 0;
    while (n > 0) {
        x = x * 10 + n % 10;           // az utolsó számjegyet hátulról fűzzük
        n /= 10;                       // és levágjuk
    }
    return x;
}

bool palindrom_szam(int n) { return n == megfordit(n); }
```

Teszt: `megfordit(1230) → 321` (a vezető 0 elvész); `palindrom_szam(1221) → igaz`; `palindrom_szam(123) → hamis`.

Módosítás: számjegyek összege, szám szóban (string), egy szám számjegyeinek maximuma.

M6 – Fájlból beolvasott jegyek: átlag és legjobb tanuló

Feladat: A `jegyek.txt` soronként „név;jegy” adatot tartalmaz. Add meg az átlagot (két tizedesre) és a legjobb tanuló nevét.

Ötlet: Beolvasás struct-ba, összegzés, maximumkiválasztás. A fájl hiányát és az üres adatot külön kezeljük; a sorvégi `\r`-t levágjuk.

```
// jegyek.txt: név;jegy soronként → átlag és a legjobb tanuló
struct Tanulo { string nev; int jegy; };

int main() {
    ifstream be("jegyek.txt");
    if (!be) { cout << "Nincs fájl" << endl; return 1; }
    vector<Tanulo> v;
    string sor;
    while (getline(be, sor)) {
        if (!sor.empty() && sor.back() == '\r') sor.pop_back();
        if (sor.empty()) continue;
        stringstream ss(sor);
        string nev, jegy;
        getline(ss, nev, ';');
        getline(ss, jegy, ';');
        v.push_back({nev, stoi(jegy)});
    }
    be.close();
    if (v.empty()) { cout << "Nincs adat" << endl; return 0; }

    int ossz = 0, mi = 0;
    for (int i = 0; i < (int)v.size(); i++) {
        ossz += v[i].jegy;
        if (v[i].jegy > v[mi].jegy) mi = i;
    }
    cout << fixed << setprecision(2);
    cout << "Átlag: " << (double)ossz / v.size() << endl;
    cout << "Legjobb: " << v[mi].nev << " (" << v[mi].jegy << ")" << endl;
    return 0;
}
```

Teszt: `Anna;5 / Bela;4 / Cili;5 → „Átlag: 4.67”, „Legjobb: Anna (5)”` (az első maximum).

Módosítás: az összes legjobb kiírása, rendezés jegy szerint csökkenő sorrendben, csak a 4-es feletti átlagú tanulók.

M7 – Betűgyakoriság

Feladat: Add meg egy szövegben minden betű előfordulását, kis- és nagybetűt azonosnak véve.

Ötlet: `map<char, int>`: a kulcs a betű, az érték a darabszám. A `map` kulcs szerint rendezett, ezért a kimenet ábécésorrendű.

```
// Betűgyakoriság (map)
void betugyakorisag(const string& szoveg) {
    map<char, int> db;
    for (char c : szoveg)
        if (isalpha((unsigned char)c)) db[tolower(c)]++;
    for (auto& p : db)
        cout << p.first << ": " << p.second << endl;
}
```

Teszt: `„Alma” → a: 2, l: 1, m: 1.`

Módosítás: a leggyakoribb betű kiírása, szavak gyakorisága, csökkenő darabszám szerinti rendezés (`vector<pair>` és `sort`).

Kódértelmezés: mit csinál ez az algoritmus?

Kódértelmezésnél ugyanúgy gondolkodj, mint a futtatásnál: vegyél egy kis példát, és kövesd **értéktáblával**, hogyan változnak a változók. Utána általánosíts egy mondatban.

K1

```
db := 0
Ciklus i := 1-től N-1-ig
  Ha T[i] > T[i+1] akkor
    db := db + 1
  Elágazás vége
Ciklus vége
Ki: db
```

Megoldás: megszámolja, hányszor **csökken** a sorozat két szomszédos eleme között. $T = [3, 1, 2, 5, 4] \rightarrow (3,1)$ igen, $(1,2)$ nem, $(2,5)$ nem, $(5,4)$ igen $\rightarrow 2$. A sorozat pontosan akkor nem csökkenő (rendezett), ha az eredmény 0.

K2

```
x := 0
Ciklus amíg N > 0
  x := x * 10 + (N mod 10)
  N := N div 10
Ciklus vége
Ki: x
```

Megoldás: megfordítja az N pozitív egész szám számjegyeit. $N = 1230: x = 0, 3, 32, 321 \rightarrow 321$ (a végi 0 a fordított szám elején eltűnik). Palindrom szám ellenőrzésére használható: $N = x$.

K3

```
Ciklus amíg b ≠ 0
  r := a mod b
  a := b
  b := r
Ciklus vége
Ki: a
```

Megoldás: két szám **legnagyobb közös osztóját** számolja (euklideszi algoritmus). $a = 48, b = 18: (48, 18) \rightarrow (18, 12) \rightarrow (12, 6) \rightarrow (6, 0) \rightarrow 6$. A ciklus akkor áll meg, ha a maradék 0.

K4

```
mi := 1
Ciklus i := 2-től N-ig
  Ha T[i] < T[mi] akkor
    mi := i
  Elágazás vége
Ciklus vége
Ki: mi, T[mi]
```

Megoldás: a **legkisebb** elem helyét és értékét adja (az első minimumét). $T = [4, 2, 7, 2, 9] \rightarrow$ hely: 2, érték: 2. A maximumkiválasztás fordítottja.

K5

```
s := T[1]
Ciklus i := 1-től N-1-ig
  T[i] := T[i+1]
Ciklus vége
T[N] := s
```

Megoldás: a tömb elemeit **eggyel balra forgatja** (az első elem a végére kerül). $T = [1, 2, 3, 4] \rightarrow [2, 3, 4, 1]$. Az s ideiglenes tárolja az elsőt, különben elveszne.

Kódmódosítási gyakorlat

A bizottság gyakran kéri, hogy egy kész programot kis változtatással átalakíts. Gyakorold ezeket az 1–3. tételek kódján:

Módosítás	Mit változtatsz
Az összegzésből legyen szorzat	A kezdőérték 1, a művelet *= . Nagy szorzatnál long long kell a túlcsordulás miatt.
Csak a pozitív elemek összege	Az összegzés elé feltétel: if (x > 0) s += x;
Átlag az összeg helyett	(double)s / t.size(); üres vektornál előbb ellenőrizd a méretet (nullával osztás).
A legnagyobb helyett a legkisebb	A reláció megfordul: t[i] < t[mi].
Az UTOLSÓ legnagyobb elem helye	A > helyett >= a maximumkiválasztásban.
Csökkenő rendezés	A rendezésben a t[j] > t[j+1] feltétel < lesz (buborék), vagy a t[j] < t[mi] feltétel > (minimumkiválasztás).
A feltétel legyen „páros ÉS 10-nél nagyobb”	if (x % 2 == 0 && x > 10) ...

10. Gyakorlás – mintatételek, értékelőlap, terv

12 saját mintatétel (A + B) · önértékelés · napi terv 2026. november 7-ig · vizsganapi lista

12 mintatétel (saját készítésű, nem hivatalos)

Húzz egyet, állíts be **30 + 20 percet**, és csináld végig a valódi menet szerint: B) feladat gépen, A) vázlat, majd hangosan, időre elmondva. Vedd fel telefonnal, és hallgasd vissza.

#	A) feladat: témakör és tartalmi szempontok	B) feladat
1	Szövegszerkesztés – stílusok és tartalomjegyzék. Stílus fogalma, bekezdés- és karakterstílus, módosítás hatása, Címsor → tartalomjegyzék, frissítés.	Összegzés: egy tömb elemeinek összege és átlaga.
2	Grafika – pixel és vektor. Jellemzők, felbontás, színmélység, fájl méret számítása, formátumok, mikor melyik.	Maximumkiválasztás: a legnagyobb elem és első helye.
3	Bemutatókészítés – tervezés. Diaminta, egységes arculat, olvashatóság, tipikus hibák.	Megszámolás: hány 10-nél nagyobb páros szám van.
4	Publikálás – HTML alapok. Dokumentum szerkezete, címsor, lista, táblázat, kép, hivatkozás, útvonalak, alt.	Eldöntés: van-e két szomszédos azonos elem.
5	Táblázatkezelés – hivatkozások és függvények. Relatív/abszolút, HA, XKERES, SZUMHATÖBB, hibák.	Kiválogatás: a páratlan elemek új sorozatba.
6	Adatbázis – relációs modell. Tábla, kulcsok, kapcsolattípusok, N:M feloldása, normalizálás.	Szétválogatás: pozitív, negatív és nulla elemek.
7	Szövegszerkesztés – hosszú dokumentum. Szakaszok, előfej és oldalszám, keresztbehivatkozás, ábrajegyzék.	Keresés: egy érték első előfordulásának helye.
8	Grafika – képszerkesztés lépései. Rétegek, kijelölés, átlátszóság, színkorrekció, átméretezés, export.	Rendezés: növekvő sorrend buborékkal, majd módosítás csökkenőre.
9	Bemutatókészítés – animáció és vetítés. Átmenet és animáció, jegyzet, előadói nézet, kimenetek.	Prímteszt: prím-e egy szám; az 1–100 közti prímek.
10	Publikálás – CSS. Szelektorok, doboz-modell, bekötés, kaszkád, reszponzív oldal.	Számjegyek: összeg és megfordítás, palindrom.
11	Táblázatkezelés – kimutatás és diagram. Mikor melyik, mezők, diagramtípusok és elemek.	Metszet: két rendezett tömb közös elemei.
12	Adatbázis – SQL. DDL/DML/SELECT, JOIN-ok, GROUP BY – HAVING, WHERE és HAVING különbsége.	Kódértelmezés: a K1–K3 feladatok egyike, vagy a szavak száma és a leghosszabb szó.

Önértékelő lap – a bizottság szemével

Értékeld a próbafeleleteidet a hivatalos rubrika szerint. Írd be a pontokat a három próba oszlopába, és nézd meg, hol csúszol.

Szempont	Max	Kérdés magadnak	1. próba	2. próba	3. próba
A) tartalom	8	Minden szempontot érintettem? Volt-e tárgyi tévedés? Mutattam gépes példát?			
B) tartalom	10	Működik a program? Helyes a tétel? Kipróbáltam szélső esetre? Tudom a módosítást?			
Logikai felépítés	4	Volt nyitómondat, sorrend, időbeosztás? A lényeget kiemelttem?			
Kifejezőkészség, szaknyelv	4	Pontos szakszavakat használtam? Érthetően magyaráztam?			
Kommunikatív készség	4	Megértettem a kérdést? Elakadáskor volt válaszm? Tudtam párbeszédet folytatni?			
Idő	–	Beleférek 20 percbe? A B) rész nem nyúlt túl?			

Napi terv: 2026. október 21. – november 7.

A gyakorlati vizsga október 20-án lesz, utána a szóbelire marad **18 nap**. A terv napi 1,5–2 óra munkára épül, szükség szerint igazítsd.

Dátum	A) téma	B) programozás	Megjegyzés
Okt. 21. (sze)	Kisokos, tétalcédulák készítése	Összegzés, megszámlolás, maximum	Könnyű nap, a gyakorlati után pihenj is.
Okt. 22. (cs)	Táblázatkezelés	Kiválogatás, eldöntés	Átfed a gyakorlatival, gyors.
Okt. 23. (p)	Adatbázis-kezelés (phpMyAdmin)	Keresés, szétválogatás	Ünnepnap, nyugodtan szánj időt a demókra.
Okt. 24. (szo)	Szövegszerkesztés	Rendezések (min., buborék)	
Okt. 25. (v)	Bemutatókészítés	Metszet, unió, bináris keresés	
Okt. 26. (h)	Grafika és képszerkesztés	M1, M2	GIMP és Inkscape gyakorlása.
Okt. 27. (k)	Publikálás (HTML, CSS)	M3, M4	A HTML-vázlat gépeld be fejből.
Okt. 28. (sze)	1. próbafelelet (2 tétel)	Javítás az értékelőlap szerint	Telefonos felvétel, utána kijavítod.
Okt. 29. (cs)	A leggyengébb 2 téma	Kódértelmezés: K1–K3	
Okt. 30. (p)	2. próbafelelet (3 tétel)	M5, M6, M7	
Okt. 31. (szo)	Szakkifejezések átnézése hangosan	Módosítási feladatok (> <=> >=, csökkenő)	
Nov. 1. (v)	Gépes bemutatók újra, hibák nélkül	Szélső esetek tesztelése	Pihenősebb nap.
Nov. 2. (h)	3. próbafelelet (tételhúzással)	Pszudokód ↔ C++ fordítás	
Nov. 3. (k)	A gyakorlati dolgozat megtekintése	Hibalista a gyenge témákra	Nézd meg, hol vesztettél pontot, és ezt pótolod a hátralévő napokon.
Nov. 4. (sze)	Gyenge témák célzott pótlása	Kódmódosítási gyakorlat időre	
Nov. 5. (cs)	Teljes próba (30 + 20 perc, valós körülmények)	Értékelés a lap szerint	Utoljára még javíts, de ne tanulj újat.
Nov. 6. (p)	Max. 1 óra átfutás	Max. 1 óra átfutás	Pihenj, készítsd ki a dolgokat, nézd át az útvonalat, korán fekjüdj le.
Nov. 7. (szo)	SZÓBELI 09:00		Zalaegerszeg, Zrínyi Miklós Gimnázium, Rákóczi u. 30., I. em. 23. terem. Ott légy 08:30-ra .

Vizsganapi lista

Előző este

- Készítsd ki: **személyi igazolvány** (vagy útlevél, jogosítvány), az **értesítő** a vizsgáról, víz, esetleg ennivaló. Segédeszköz a források szerint nem kell; ha mégis szeretnél vinni, az értesítőn szereplő „vizsgaszabályzat által megengedett segédeszközöket” érdemes az iskolában tisztázni.
- Nézd meg az útvonalat Keszthelyről Zalaegerszegré (szombat reggel), a parkolást és az érkezési időt. Számolj tartalékkal, hogy 08:30-ra ott legyél.
- Egy könnyű átfutás a nyitómondattal, a pszeudokód–C++ táblázattal és a gyakori hibákkal. Utána pihenj.

A vizsga napján

- Reggelizz, érkezz időben, és **kapcsold ki a telefont**.
- Tétel kihúzása után **először olvass**: számozd meg az A) szempontokat, értsd meg a B) feladatot, és csak utána kezdj.
- **B) gépen előbb**, A) vázlat utána. Mentsd el, és futtasd le, mielőtt bemész.
- A felelet elején **nyitómondat**, a végén egymondatos összegzés. Hangosan gondolkodj a gépnél.
- Ha elakadsz, mondd el, amit tudsz, és kérj segítséget a kérdés pontosításához. Maradj nyugodt: a bizottság vezetni fog.

Sok sikert! A szóbelire a legfontosabb a rendszeres, hangos gyakorlás: minél többször mondd el időre, annál kevesebb dolog lep meg.